Hidden in the Leftovers

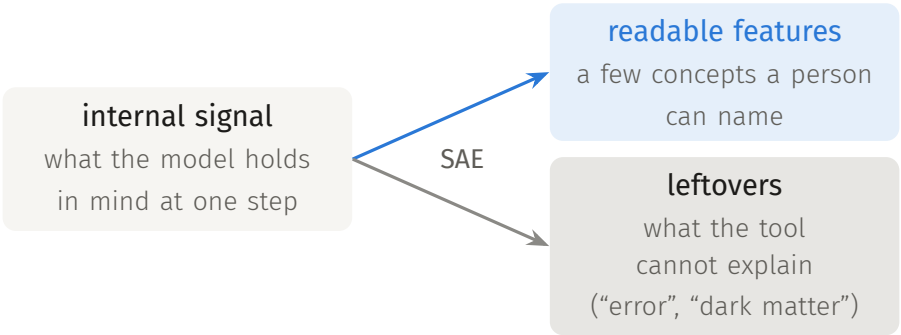
Mapping the task-specific blind spots of sparse autoencoders in English and Indian languages

[Student name]

October 2026

[Course name]

We read language models with sparse autoencoders



Sparse autoencoder (SAE): the main tool for reading a model’s internal signal; used to explain, audit and steer models [1, 2].

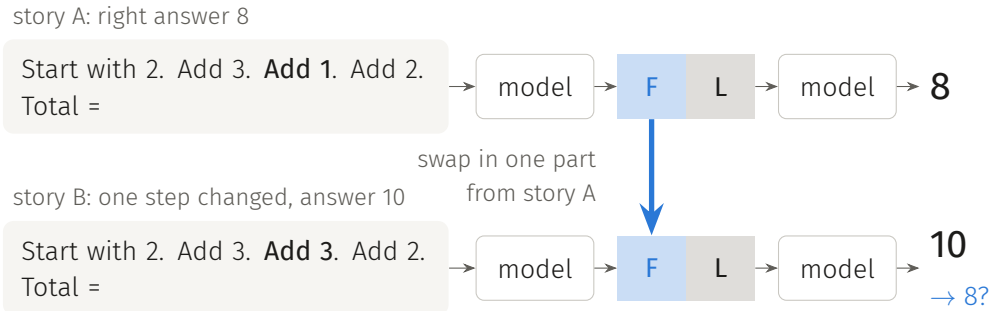
98–99% of the model’s language ability survives the SAE on ordinary text, so we trust it.

The worry: does the real work hide in the leftovers?

An average can hide what matters.

- A popular idea: the part SAEs cannot explain, their “dark matter” [3], is where the model’s real computation lives.
- If true, explanations built from SAEs miss exactly the part an audit cares about.
- We tested it directly, on puzzles where the model must keep track of something.

The swap test: does the right answer come back?



F = readable features, L = leftovers. Score: **how much of the right answer comes back**, from 0% (still 10) to 100% (back to 8). Swapping everything gives 100%.

Pilot: four puzzles the model can solve

Running sum

`Start with 2. Add 3. Add 1. Add 2. Total =` 8

keeps a total

Object tracking

`The key is in box B. … The key moves to box C. … The key is in box` C

links objects to boxes

Variable copying

`t = 7. q = 4. x = q. q = 2. x =` 4

follows an assignment

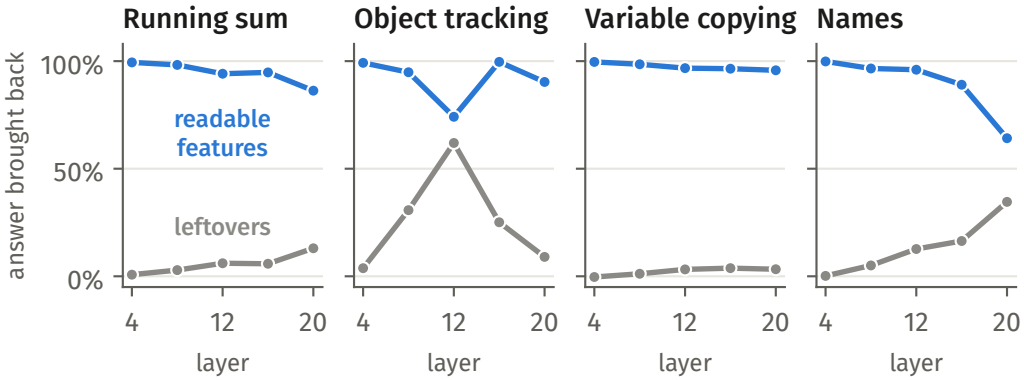
Names (control)

`When Mary and John went to the store, John gave a drink to` Mary

a well-studied case [6]

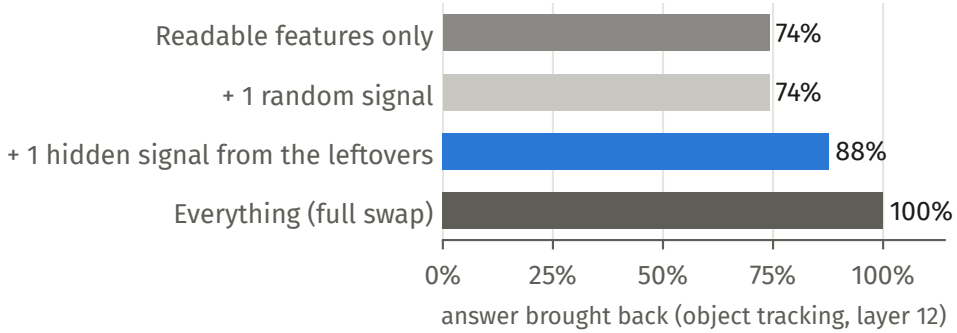
Model: gemma-2-2b (Google) · SAEs: Gemma Scope [5] · 150 test pairs per puzzle · three harder puzzles were dropped: the model could not solve them.

Mostly no: the readable features carry most of the answer



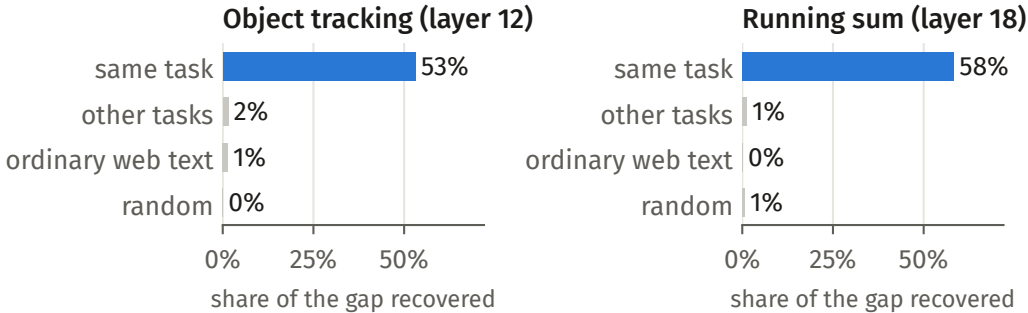
The leftovers alone bring back little. One exception stands out: object tracking, middle layers.

Near-perfect on average, yet 26% missed on one task



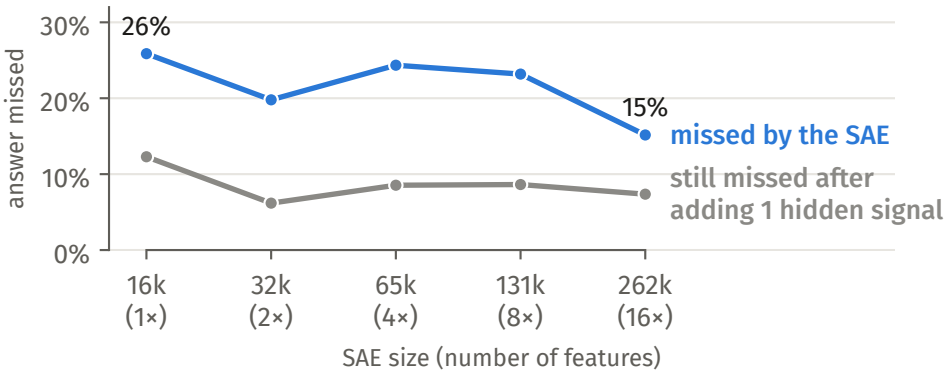
The same SAE keeps 98–99% on ordinary text. Here it misses 26%, and half of that comes back with **one** hidden signal from the leftovers. A random signal adds nothing.

The hidden signal belongs to the task



Signals learned from the same task fill the gap. Signals from other tasks or ordinary web text do almost nothing.

A 16× bigger SAE shrinks the gap but does not close it



At every size, one hidden signal still recovers about half of what is missed.

Three ways these tests fool you (we hit all three)

Swap the whole sentence, not just the last word

47% vs 100%

object tracking, layer 12: last word only vs from the first changed word

Check the words don't give the answer away

100%

a reader that sees only the words, no model, already gets variable copying and names right

Readable is not the same as used

84% vs 47%

the box can be read from the leftovers, yet swapping there brings back at most 47%

So: SAEs have task-specific blind spots

Averages cannot see them.

They have to be checked task by task, and language by language.

Q1

Which tasks have blind spots?

more kinds of tasks; how big, how specific

Q2

What is the hidden signal?

is it how the model links “what goes with what”?

Q3

Are they bigger in Indian languages?

Hindi and Hinglish first

Q1 and Q2: which tasks, and what is the hidden signal?

Kind of task	Examples	Status
Linking things	objects in boxes; people and their cities (new)	pilot + new
Keeping a count	running sum; longer chains (new)	pilot + new
Following references	variable copying	pilot
Control	names	pilot

Q1: for every task, how much does the SAE miss, is it task-specific, does a bigger SAE fix it? Do two linking tasks share the same hidden signal?

Q2, binding test: models link objects to attributes with dedicated signals [4]. Change only the hidden signal: does the model now name the other box?

Q3: are the blind spots bigger in Indian languages?

English (pilot baseline)

Hindi

Hinglish (Hindi, Latin script)

Tamil or Bengali

French (non-Indian control)

The SAEs were trained on mostly English text [5].

Three checks per language

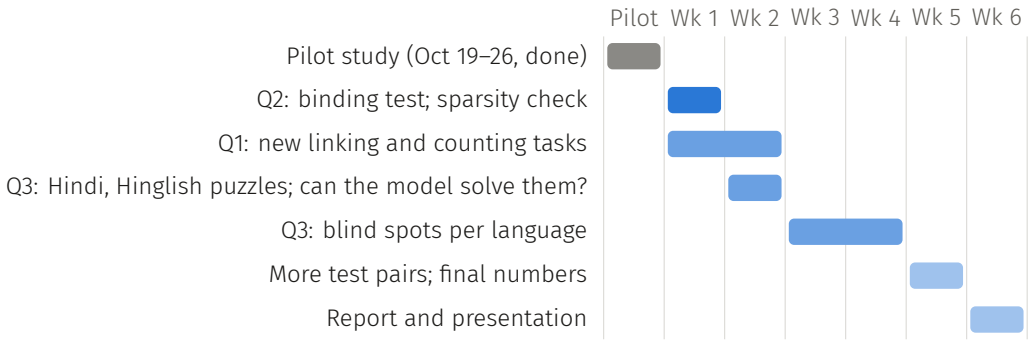
1. Does the SAE explain Hindi text as well as English?
2. Is the blind spot bigger in Hindi?
3. Is it the same hidden signal? (learn it in English, test it in Hindi)

Same puzzles, translated (illustrative, to be checked)

शुरुआत 2 से करें। 3 जोड़ें। 1 जोड़ें। 2 जोड़ें। कुल = 8

`Jab Rahul aur Priya market gaye, Priya ne ek book di. Book kisko mili? Answer:` Rahul

Plan: six weeks, two deliverables



Deliverables: a blind-spot map (tasks × layers × SAE sizes × languages) and the swap test as a cheap check before trusting an SAE explanation.

In one line

Sparse autoencoders look near-perfect on average, but they hide task-specific blind spots in their leftovers.

- **Found:** object tracking loses 26%; half is one hidden signal that only that task recovers, and a 16× bigger SAE does not remove it.
- **Proposed:** map these blind spots across tasks and in Hindi and Hinglish, and test what the hidden signal is.

Backup: the original claims and what the pilot found

Original claim	Pilot verdict
The leftovers carry what the model tracks	**Mostly no:** readable features bring back 74% to 100%; leftovers 0% to 13% (object tracking up to 62%)
The tracked state sits in the leftovers in a simple, readable shape	**Readable, not used:** a reader decodes it there (84%), but the model does not use it at that spot yet
A few hidden signals in the leftovers fill the gap	**Yes, narrowly:** one signal recovers about half of object tracking's 26% gap; only for that task

Backup: caveats

- One model (gemma-2-2b), one SAE family (Gemma Scope), made-up puzzles.
- The names control also loses 24% to 36% in late layers, but that loss is spread over about 32 signals, so part of the late-layer loss is a general SAE weakness.
- Object tracking's "leftovers only" score (62% at layer 12) is inflated: replacing the leftovers with noise already gives 51%.
- The 131k and 262k SAEs also use more features per word (124 to 139 vs 75 to 89 at 16k), so part of the shrinking gap may come from that, not from size.
- The binding explanation and everything multilingual are plans, not results.

References i

[1] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. Transformer Circuits Thread, 2023. URL https://transformer-circuits.pub/2023/monosemantic-features/index.html. Published October 4, 2023.

[2] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023. URL https://arxiv.org/abs/2309.08600.

[3] Joshua Engels, Logan Riggs Smith, and Max Tegmark. Decomposing the dark matter of sparse autoencoders. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=sXq3Wb3vef.

[4] Jiahai Feng and Jacob Steinhardt. How do language models bind entities in context? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=zb3b6oKO77.

[5] Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 278–300, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.blackboxnlp-1.19. URL https://aclanthology.org/2024.blackboxnlp-1.19/.

[6] Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=NpsVSN6o4ul.